EXPERIMENT - 51

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, arr[]. Pass the array to a function named next_greater() using its address. Use pointers to pass the array between the calling function and next_greater(). The function next_greater() must:

- Find the next greater element for each element in arr[].
- Search only the elements to the right of the current element and select the nearest element that is greater than the current element.
- Display -1 if no greater element exists to the right.
- Display the results in the order of their appearance in the array, separated by commas.
- Use nested loops.

For 4 5 2 25 the output is 5, 25, 25, -1. For 13 7 6 12 it is -1, 12, 12, -1. For 1 3 2 4 it is 3, 4, 4, -1. For 1 2 3 4 5 it is 2, 3, 4, 5, -1.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and declare the variable length array arr of size n.

Step 3: Read the n elements of arr.

Step 4: Call next_greater(&arr[0], n).

Step 5: STOP

Function next_greater()

Step 1: Repeat for i = 0 to n − 1:

Set found = −1.

Repeat for j = i + 1 to n − 1: if arr[j] is greater than arr[i], set found = arr[j] and leave the inner loop.

If i is not 0, display a comma and a space.

Display found.

Step 2: Move to a new line.

Step 3: Return to the calling function.

3. Test Case Table

#	Array	Expected Output	Actual Output	Result
1	4 5 2 25	5, 25, 25, -1	Matches example 1 in the question	**PASS**
2	13 7 6 12	-1, 12, 12, -1, since nothing beats 13	Matches example 2	**PASS**

#	Array	Expected Output	Actual Output	Result
3	1 3 2 4	3, 4, 4, -1, where 1 takes the nearer 3 and not 4	Matches example 3	**PASS**
4	1 2 3 4 5	2, 3, 4, 5, -1, each one taking its neighbour	Matches example 4	**PASS**
5	5 4 3 2 1	Five −1 values, as the array only falls	−1 five times	**PASS**
6	2 2 2	Three −1 values, since equal is not greater	−1 three times	**PASS**
7	10	−1, as there is nothing to the right at all	−1	**PASS**

4. Source Code

nextgreater51.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

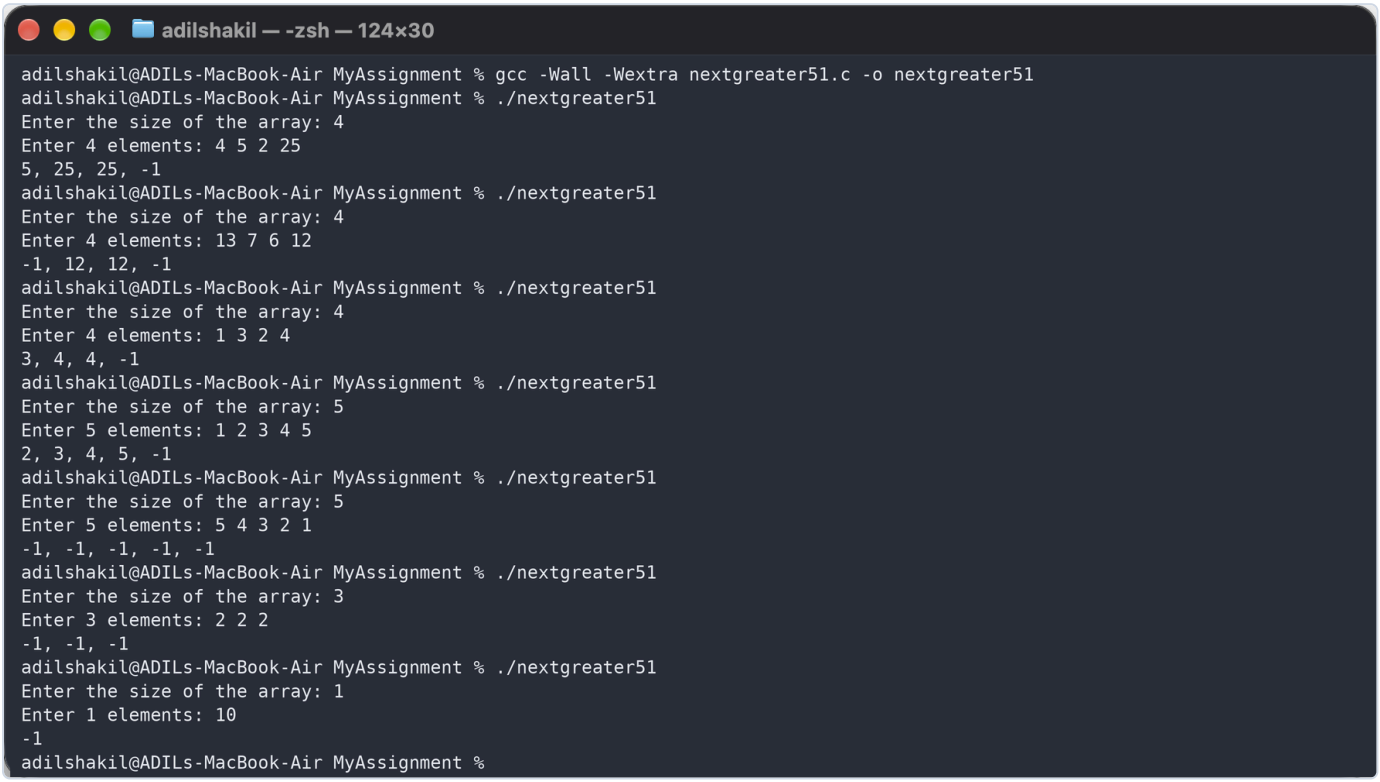
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

5. Compilation and Output

Compiled with `gcc -Wall -Wextra nextgreater51.c -o nextgreater51` so the small warnings would show up too. It built clean.

All Seven Runs



All seven cases came out right, and the first four are the examples given in the question.

next_greater() takes `int *arr`, so it is handed the address of the array and not a copy of it, and the call is written as `&arr[0]`.

The question asks for nested loops and that is what the job needs anyway. The outer loop picks one element and the inner loop hunts for its answer. The inner loop starts at `i + 1` rather than 0, which is the only thing making it look to the right.

The `break` is the part I would have got wrong without the examples. Nearest means the first greater one, not the biggest one. In 1 3 2 4 the answer for 1 has to be 3, and without the break the loop would carry on, find 4 as well, and leave 4 sitting in found.

found is set back to −1 at the top of every round, so when the inner loop finds nothing the −1 is already there and no extra check is needed afterwards.

The comma goes before each answer instead of after it, which keeps the line from ending with a stray comma.

Cases 5, 6 and 7 I added myself. A falling array where every answer is −1, an array of equal values, and a single element. Case 6 is the useful one, because it checks that the test is `arr[j] > arr[i]` and not >=, so an equal value is not counted as greater.